

Department of Artificial Intelligence and Data Science

LogSight: Multi-View Anomaly Detection & Root-Cause Localization

Dr. Suresh Kumar S
Professor
Mrs. Jayasri Archana Devi
Assistant Professor

Jegadeeswaran D (231801069)
Karthick S(231801079)
Koushal V(231801084)

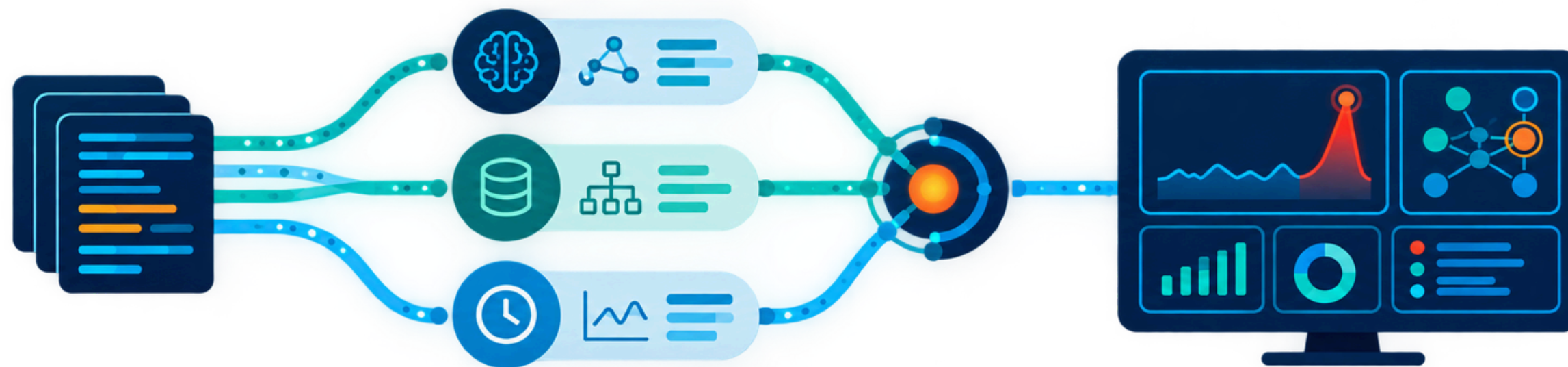
PROJECT OVERVIEW

Problem: Distributed systems (microservices, Kubernetes, cloud) produce huge volumes of unstructured logs. Failure diagnosis is still manual and reactive, and existing tools rely on thresholds and give little explanation.

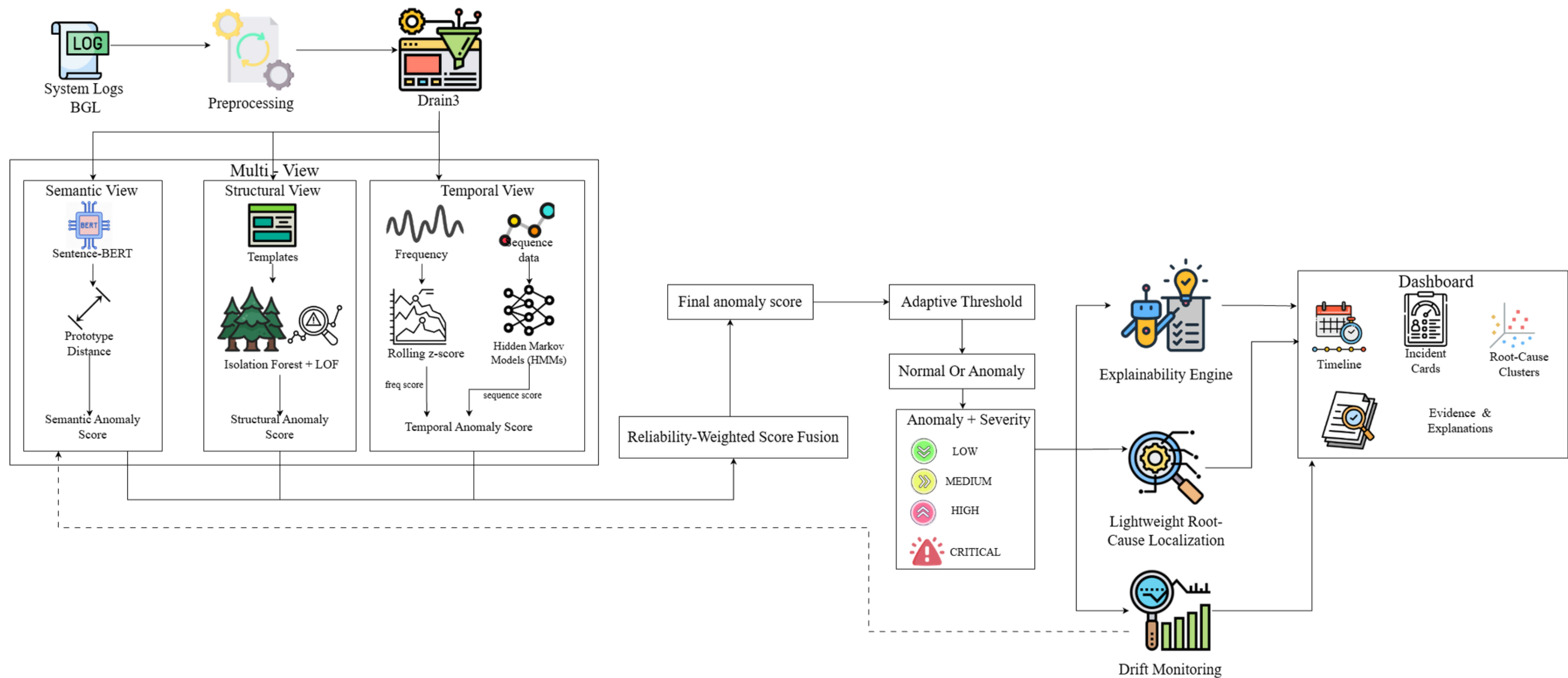
Proposed solution: An unsupervised framework that learns normal system behaviour and detects known and previously unseen anomalies without labeled failure data.

- Three complementary views of each log event:
 - Semantic: Sentence-BERT embeddings with KMeans prototypes
 - Structural: Drain3 templates, log level and parameters, scored with Isolation Forest and LOF
 - Temporal: frequency, burst rate and sequence order, scored with an HMM
- Reliability-weighted fusion: The three view scores are combined using each view's reliability, not just its score magnitude.
- Adaptive thresholding and drift monitoring: A rolling median and MAD threshold, plus a KS test, separate genuine anomalies from legitimate changes in normal behaviour.

- Explainability: Every anomaly comes with evidence: per-view deviation, the nearest normal example, the preceding events and a severity level (Low to Critical).
- Lightweight root-cause localization: Anomalies are grouped into incidents, and candidate components are ranked by correlation, with no service-dependency graph needed.
- Dashboard: Shows the anomaly timeline, incident cards and root-cause clusters.
- Datasets and evaluation: BGL (primary), HDFS (secondary) and Thunderbird (optional), using Precision, Recall, F1, AUC-ROC and AUC-PR. The evaluation includes an ablation study and a test on unseen anomaly patterns.



SYSTEM ARCHITECTURE



MODULES

Module 1: Setup, Ingestion and Cleaning

- **What:** Python environment, project structure, and loading and cleaning the raw BGL logs.
- **Why:** A reproducible base, and clean input so malformed rows don't break later stages.
- **Explanation:** All required libraries are installed and a fixed random seed is set. Lines are split on whitespace runs, 10 malformed rows are dropped, and the data is sorted chronologically. The result is saved as Parquet (4,713,483 rows), which keeps datatypes and loads faster than CSV.
- **Status:** Done

Module 2: Exploratory Analysis and Log Parsing

- **What:** EDA on the raw logs, Drain3 template parsing, and EDA on the parsed output.
- **Why:** To validate the data, convert free text into structured templates, and find weaknesses early.
- **Explanation:** The raw data shows a 7.39% anomaly rate (matching published BGL figures) and bursty arrivals (median gap 26 ms). Drain3 found 1,819 templates, within the expected range. 63.83% of templates are singletons, mostly high-entropy register dumps. This is a documented limitation, compensated by the semantic and temporal views.
- **Status:** Done

MODULES

Module 3: Multi-View Feature Construction

- **What:** Builds the semantic, structural and temporal feature views for every event.
- **Why:** Analysing each event from complementary angles is the core idea of the project.
- **Explanation:** All views are row-aligned at 4,713,483 rows with no NaNs.
 - **Semantic:** 384-dimensional SBERT embeddings, computed once per unique template.
 - **Structural:** template ID, parameter count, level rank, component, type and template frequency.
 - **Temporal:** time since previous event, rolling frequency, burst rate, novelty, transition probability, entropy and repeated-event ratio.
- **Status:** Done

Module 4: Per-View Scoring and Fusion

- **What:** An anomaly score for each view, combined into one final score.
- **Why:** Each view catches a different failure type, and reliability-based weighting avoids circular score averaging.
- **Explanation:**
 - **Semantic:** KMeans prototypes, scored as 1 minus the maximum cosine similarity.
 - **Structural:** Isolation Forest and LOF combined.
 - **Temporal:** HMM likelihood plus a rolling z-score.
 - **Fusion:** Each view gets a reliability signal separate from its score, and a softmax turns these into weights.
- **Status:** Not started

MODULES

Module 5: Adaptive Detection, Severity and Drift

- **What:** Decides which scores are anomalies, how serious they are, and whether behaviour has shifted.
- **Why:** Fixed thresholds fail when normal behaviour changes, and drift can cause false alarms.
- **Explanation:** The threshold is a rolling median plus a scaled MAD. Severity (LOW to CRITICAL) combines score, persistence, blast radius and frequency. A KS test flags drift when $p < 0.01$ for 3 consecutive windows.
- **Status:** Not started

Module 6: Explanation and Root-Cause Localization

- **What:** Explains each anomaly and ranks likely originating components.
- **Why:** Engineers need to know why an event was flagged and where the problem started.
- **Explanation:** Each anomaly gets an evidence package (per-view deviations, nearest normal example, preceding events), with an optional LLM rewrite into plain language. Root-cause localization clusters anomalies into incidents by time window, computes component co-occurrence, groups by template signature, and ranks candidates. No service-dependency graph is needed.
- **Status:** Not started

MODULES

Module 7: Evaluation and Dashboard

- **What:** Measures detection quality and presents the results in an interactive dashboard.
- **Why:** It validates the multi-view approach and makes the results usable by non-experts.
- **Explanation:** Metrics are Precision, Recall, F1, AUC-ROC and AUC-PR on a chronological split, plus recall on unseen templates. Ablation runs compare single-view, two-view and full configurations. The Streamlit dashboard shows the anomaly timeline, component risk, incident cards and root-cause clusters from pre-computed results.
- **Status:** Not started

TECH STACK

PROGRAMMING & DATA PROCESSING

- Python
- Pandas, NumPy, SciPy
- Drain3 - Log parsing

Machine Learning & Anomaly Detection

- Sentence-BERT - Semantic embeddings
- Scikit-learn - KMeans, Isolation Forest, LOF
- HMMlearn - Temporal analysis

Detection & Analysis

- Adaptive Score Fusion
- Median/MAD Thresholding
- KS Test - Drift Detection
- Clustering & Correlation — Incident Localization

Evaluation

- Precision, Recall, F1
- AUC-ROC, AUC-PR
- Ablation & Unseen-Anomaly Testing

Dashboard

- React.js - Interactive Dashboard
- Charts & Incident Views —
Visualize anomalies, evidence, and
RCA results

ABSTRACT

Modern systems generate a huge number of log messages, making it difficult to identify failures and understand where they started. This project develops an unsupervised log analysis framework that learns normal system behavior and detects unusual and previously unseen patterns by looking at the meaning, structure, and behavior of logs. The system also explains why an event was considered abnormal, identifies the components that are likely involved, and presents the results through a dashboard to help engineers investigate problems more easily.

Keywords: System Logs • Anomaly Detection • Unsupervised Learning • Multi-View Analysis • Explainable AI • Root-Cause Localization

Thank You
